Лабораторная работа № 1.

Создание виртуального окружения. Контроллеры и маршруты

Цель работы: настроить виртуальное пространство. Разработать контроллер приложения и организовать маршруты

Методика выполнения

1. Создание виртуального окружения

1 способ

Создадим папку MyProject, и в этой папке создадим проект с названием вашего проекта, например MySite и создадим виртуальное окружение.

Установим виртуальное окружение с помощью модуля vnv, который установлен в python и вводим команду **python -m venv venv**, где название виртуального окружения venv, через терминал проекта.

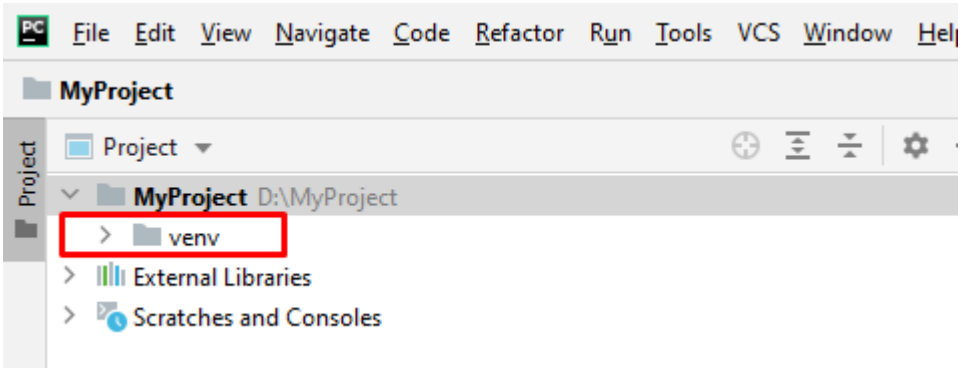


Рисунок 1

В папке MyProject появится папка **venv** вашего виртуального окружения.

Введите команду pip list. Выведится список глобально установленных библиотек.

Возможно выведится строка с рекомендацией обновления pip (зеленая строка на рисунке 2)

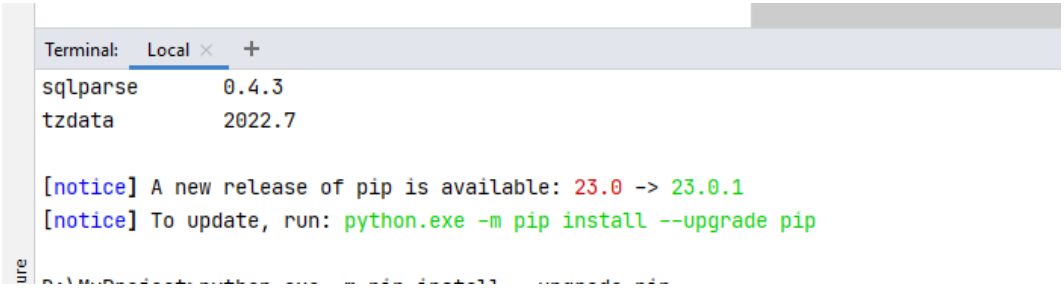
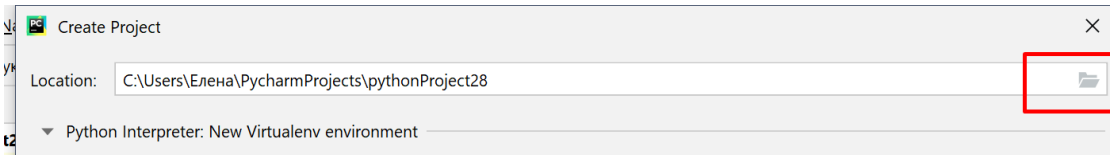


Рисунок 2

Рисунок 1



Если просмотрим вложенные файлы, то увидим те пакеты, которые установлены на данный момент.

Например, в папке Scripts находится python.

2 способ

Второй способ основан на автоматическом создании средствами системы программирования PyCharm, которое создается при создании проекта.

После создания виртуального окружения, необходимо его активировать. Для этого необходимо перейти в Терминал PyCharm и набрать команду venv\Scripts\activate

Файл activate, который находится в папке Scripts активирует виртуальное пространство (рисунок 3).

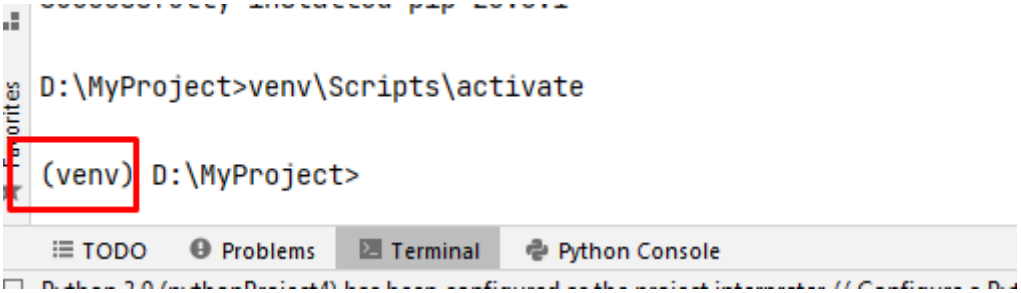


Рисунок 3

Как вы видите (рисунок 3), в командной строке стоит не только путь нахождения папки проекта, но и имя нашего виртуального окружения.

Запустим команду pip list

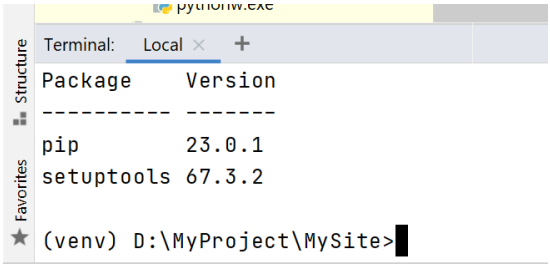


Рисунок 4

Данная команда указала на наличие двух пакетов.

```
(venv) D:\MyProject>pip list
Package    Version
---------- -------
pip        20.2.3
setuptools 49.2.1
WARNING: You are using pip version 20.2.3; however, version 23.0.1 is available.
You should consider upgrading via the 'd:\myproject\venv\scripts\python.exe -m pip install --upgrade pip' command.

(venv) D:\MyProject>
```

Рисунок 5

Следующим шагом является установка Django

Установочный пакет Django находится по адресу pypi.org (рисунок 6)

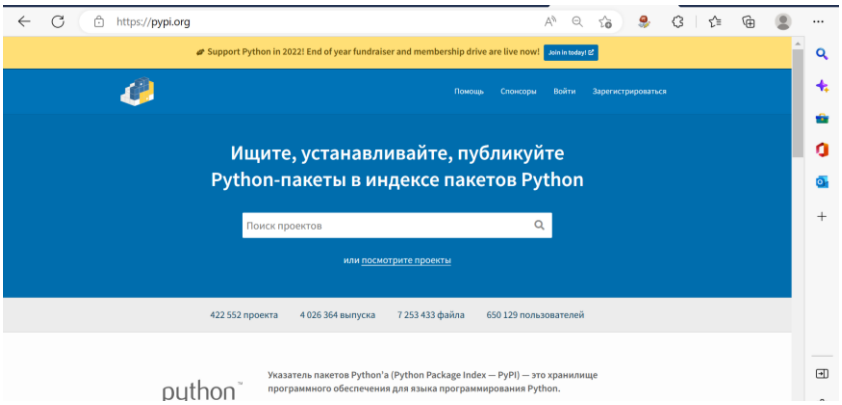


Рисунок 6

В поисковой строке введите Django и в списке выберете последнюю версию (рисунок 7)

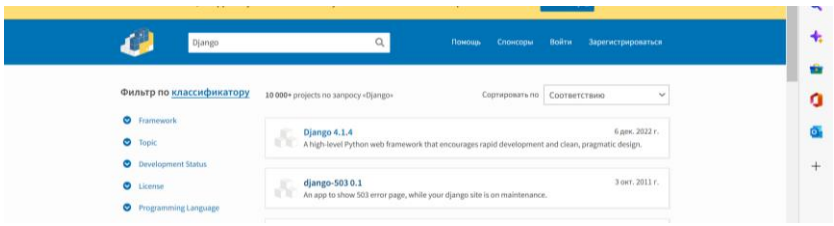


Рисунок 7

Например, Django 4.1.4.

Добавим Django в наш проект в виртуальном окружении, созданном ранее. Открываем через PyCharm. И используем команду pip install Django.

Снова выполним команду pip list

```
(venv) D:\MyProject>pip list
Package    Version
---------- -------
asgiref    3.6.0
Django     4.1.7
pip        20.2.3
setuptools 49.2.1
sqlparse   0.4.3
tzdata     2022.7
WARNING: You are using pip vers
```

Рисунок 8

Мы видим обновленный список пакетов, в том числе, Django

После установки можно приступать к созданию проекта. Но прежде всего давайте разберем команды, которые нам могут пригодиться.

1. django-admin

Это команда получения списка доступных подкоманд

```
Type 'django-admin help <subcommand>' for help on a specifi

Available subcommands:

[django]
    check
    compilemessages
    createcachetable
    dbshell
    diffsettings
    dumpdata
    flush
```

Рисунок 9

Многие команды из этого списка мы будем с вами использовать при работе над проектом.

Начнем работу с команды старта проекта – startproject.



Рисунок 10

В терминале введем команду

django-admin startproject MySite,

где MySite – название вашего проекта

После ввода этой команды, в дереве вашего проекта появится папка с названием MySite.

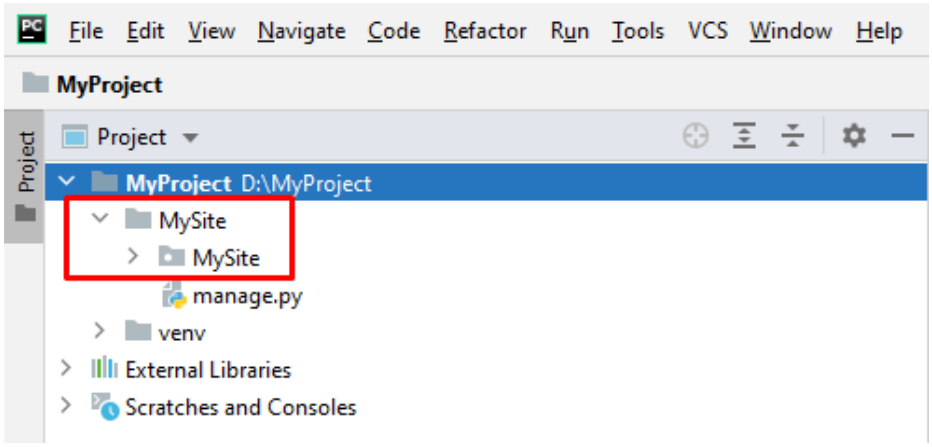


Рисунок 11

Внутри этой папки создана одноименная папка, в которой сформирован пакет, задающий конфигурацию пакета. О том, что это пакет, говорит нам папка _init_py.

Одноименную папку называют пакетом конфигурации.

Проект готов, он, конечно, еще пустой, но мы можем его попробовать запустить. Нужно запустить сервер и для этого мы обращаемся к manage.py.

Для этого перейдем в папку проекта с помощью команды

cd MySite

и здесь уже команду (рисунок 13)

python manage.py runserver



Рисунок 13

Команда runserver запускает тот самый отладочный веб сервер который поставляется вместе с Django, т.е. дополнительно устанавливать не надо. По умолчанию он запускается по адресу http://127.0.0.1:8000/

Если перейдем по этому адресу, то уже запустится шаблон нашего будущего сайта (рисунок 14).

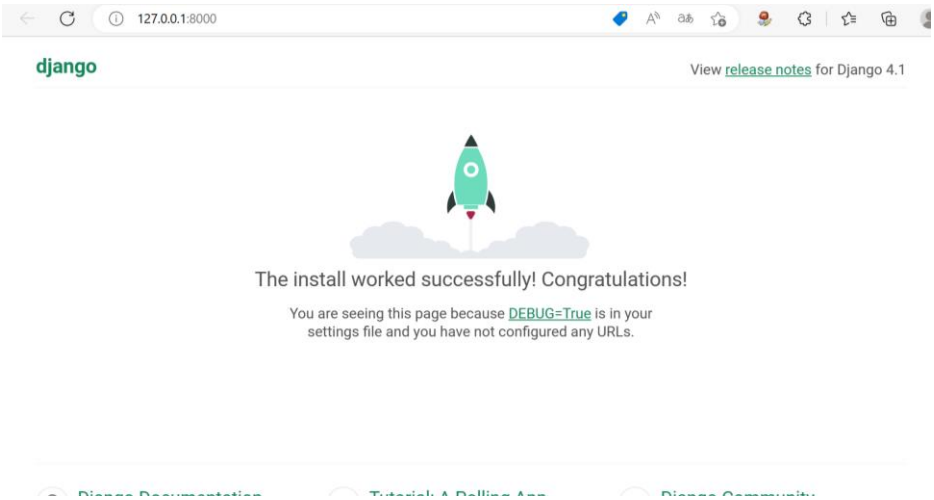
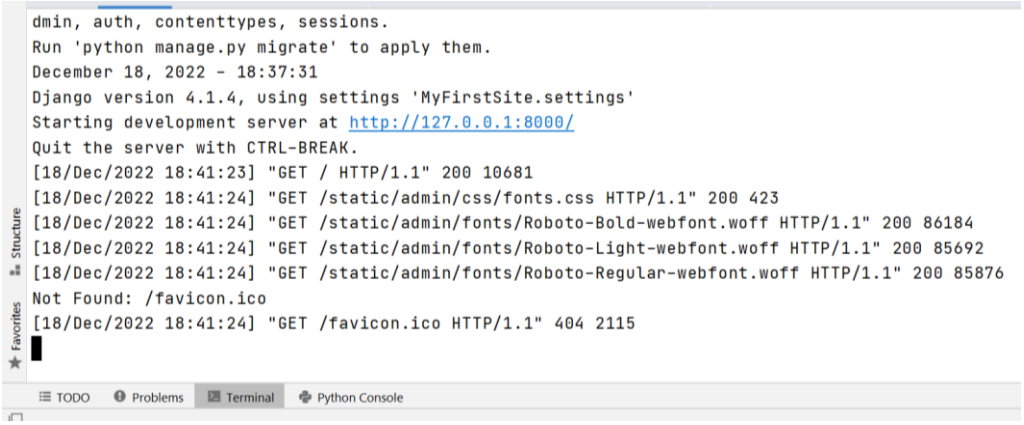


Рисунок 14

Здесь есть http запросы



Все то, что мы запрашиваем в Django, он отображает все запросы здесь с терминале (рисунок 15).

Прелесть отладочного сервера, что он проверяет изменения в проекте и если вы что-то поменяли в существующем коде, он перезапустит сервер и подхватит эти изменения, но не поддерживает проверку в новых файлах.

Для перезагрузки отключить manager.pty с с помощью комбинации клавиш Ctrl+C.

После запуска проекта появился новый файл (рисунок 16).

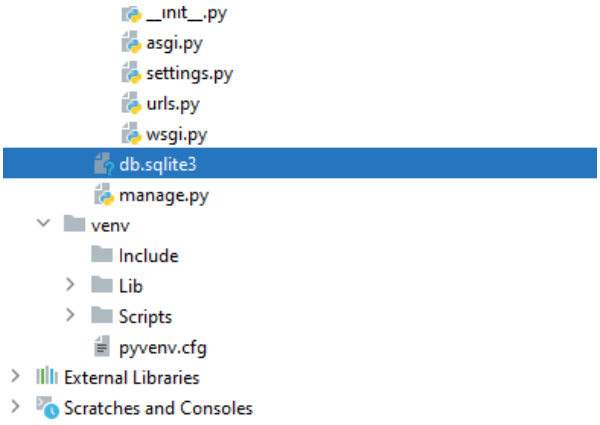


Рисунок 16

Это файл базы данных sqlite, которая поддерживается Django по умолчанию. Но Django поддерживает и другие БД.

Проект пустой пока, нет никакого функционала. Все что мы видим сейчас это пакет конфигурации проекта. Чтобы создать что-нибудь, необходимо создать хотя бы одно приложение. И весь сайт состоит из приложения.

Создадим новостной сайт. Можно открыть вторую вкладку терминала и в командной строке терминала перейдем в папку MySite, где находится manage.py и введем команду cd MytSite

И введем команду создания приложения

python manage.py startapp news

где news – это имя создаваемого приложения

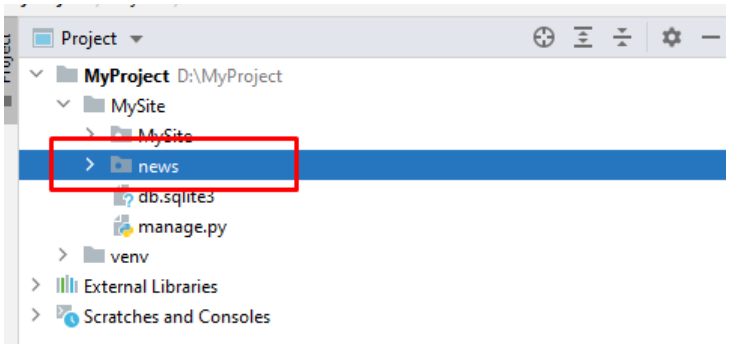


Рисунок 17

После создания приложения, чтобы Django корректно работало, это приложение необходимо зарегистрировать. Регистрируется в конфигурации проекта в файле settings.py

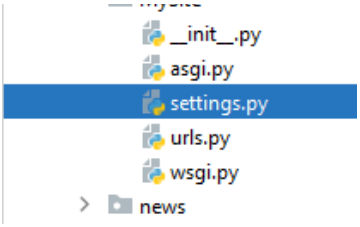


Рисунок 18

В этом файле хранятся всевозможные настройки и регистрируют в части INSTALLED_APPS

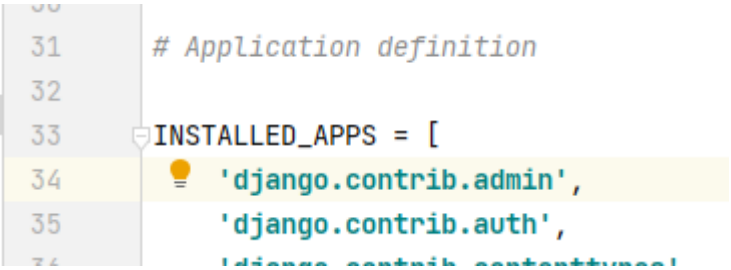


Рисунок 19

Здесь необходимо указать путь к пакету файла конфигурации app.py и указать класс

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'news.apps.NewsConfig',
]
```

Рисунок 20

Для создания контроллеров используется файл views.py

Напишем первую контроллер (функцию), которую назовем index. На вход эта функция принимает обязательный аргумент – экземпляр класса HTTP request и называется он request. В нем хранятся данные о полученном запросе, о его клиенте, его браузере, хранятся куки, сессии и т.д. Когда мы получили запрос, то нам нужно вернуть какой-то ответ.

Возвращать будем HTTP ответ, который нам позволит сформировать класс HTTP respons. Его необходимо импортировать

```
from django.http import HttpResponse
```

и будем его же возвращать и в качестве аргумента передадим строку.

```
from django.shortcuts import render
from django.http import HttpResponse


def index(request):
    print(request)
    return HttpResponse('Hello world')
```

Рисунок 21

Если попытаемся набрать в адресной строке браузера адрес нашего приложения news, то мы получим ошибку, поскольку маршрутизатор не знает такой адрес. Page not found at /news/ , то получим ошибку 404 (рисунок 22)

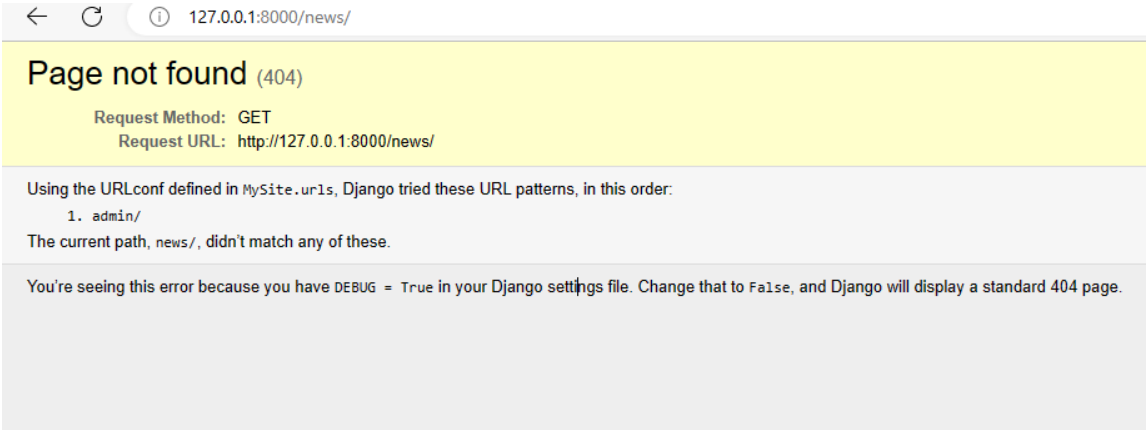


Рисунок 22

Django должен понимать, что при запросе того или иного адреса он должен обратиться к данному приложению, именно к данному контроллеру функции.

Здесь на помощь приходит маршрутизатор, который принимает первичный запрос клиента и сравнивает его со списками записанных маршрутов, адресов и говорит, есть ли у него такой адрес или нет. Если есть, то он вызывает этот адрес и говорит клиенту, куда ему надо отправиться.

Список адресов находится в пакете конфигурации в файле urls.py

Здесь уже есть один адрес админки

```
import ...

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

Рисунок 23

Если мы в адресной строке браузера допишем admin (http://127.0.0.1:8000/admin), то получим страницу входа в админку

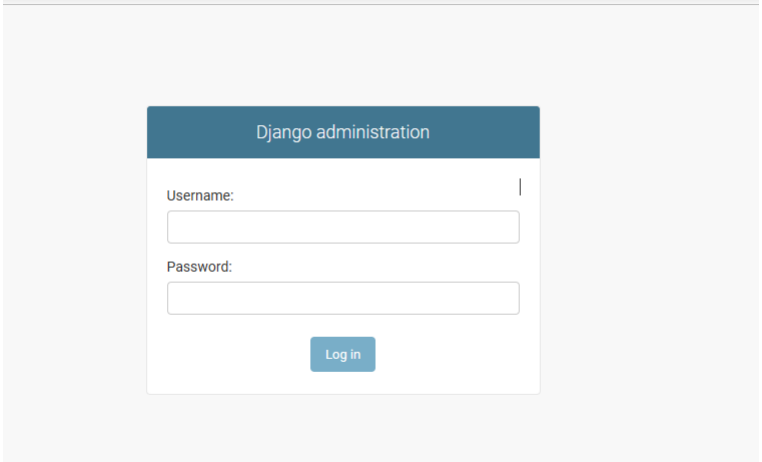


Рисунок 24

Для того, чтобы прописать маршрутизатору путь необходимо импортировать функцию index контроллера views

```
from news.views import index
```

При этом, как мы видим, на рисунке красным цветом подсвечиваются ошибки. Это означает, что не знает такого пути, потому что корневой директорией для нашего проекта является верхняя папка MySite, в которой ищется папка views и не находит. Но эта папка есть во вложенной папке MySite и поэтому эту папку надо указать как корневую для нашего проекта.

```
from news.views import index

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

Рисунок 25

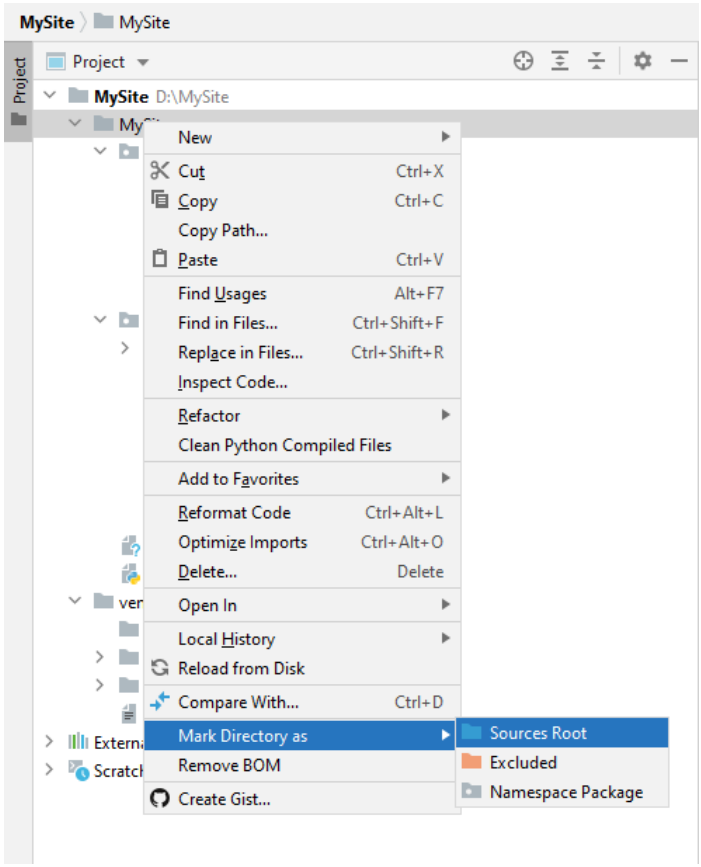


Рисунок 26

Для этого правой кнопкой мыши по этой папке щелкаем и выбираем Mark Directory as/Sources Root.

Мы импортировали нашу функцию index из модуля news в модуль views. Теперь можем вызвать функцию path с помощью этой функции мы можем указать новый маршрут. Таких маршрутов может быть много. Но нас пока интересует один. Что делает маршрутизатор, получив этот адрес? Он проходит по списку маршрутов и ищет совпадения. Первое найденное совпадение его устраивает, и он вызывает нужную функцию. В данном случае функция index, которую только что импортировали.

После импортирования вводим функцию path, которая указывает маршрут.

path('**news/**', index)

Маршрутизатор отбрасывает весь путь кроме имени нашего приложения **news**.

В терминале можно увидеть, выведенный нами, запрос.

Если введем строку print(dir(request)) и после обновления страницы index просмотрим в Терминале все методы этого запроса

```
def index(request):
    print(dir(request))
    return HttpResponse('Hello wordl')
```

Рисунок 27

```
Django version 4.1.7, using settings 'MyFirstSite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
['COOKIES', 'FILES', 'GET', 'META', 'POST', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash
, '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', '_current_s
get_raw_host', '_get_scheme', '_initialize_handlers', '_load_post_and_files', '_mark_post_parse_error', '_messages', '_read_started', '_set_content_type_params', '_set_post', '_
'body', 'build_absolute_uri', 'close', 'content_params', 'content_type', 'csrf_processing_done', 'encoding', 'environ', 'get_full_path', 'get_full_path_info', 'get_host', 'get_
', 'parse_file_upload', 'path', 'path_info', 'read', 'readline', 'readlines', 'resolver_match', 'scheme', 'session', 'upload_handlers', 'user']
[16/Feb/2023 23:14:13] "GET /news/ HTTP/1.1" 200 11
```

Рисунок 28

Давайте создадим еще одну функцию test

```
def index(request):
    # print(dir(request))
    return HttpResponse('Hello wordl')

def test(request):

    return HttpResponse('<h1>Тестовая страница<\h1>')
```

Рисунок 29

И тоже для него зададим маршрут.

```
15  """
16  from django.contrib import admin
17  from django.urls import path
18  from news.views import *
19
20  urlpatterns = [
21      path('admin/', admin.site.urls),
22      path('news/', index),
23      path('test/', test),
24  ]
```

Рисунок 30

Посмотрим на нашу страницу. И распечатаем наш запрос

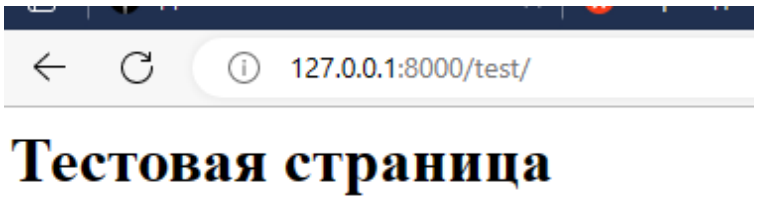


Рисунок 31

Страница test должна быть вложенной для приложения news.

Перепишем маршрут следующим образом:

Для этого нужно хранить все маршруты в нашем приложении news. Достигается путем импорта этих маршрутов. Для этого импортировать функцию include

```
15  """
16  from django.contrib import admin
17  from django.urls import path, include
18
19  from news.views import *
20
21  urlpatterns = [
22      path('admin/', admin.site.urls),
23      path('news/', include('news.urls')),
24  ]
25
```

Рисунок 32

Этого файла в нашем приложении нет, поэтому мы его должны создать .

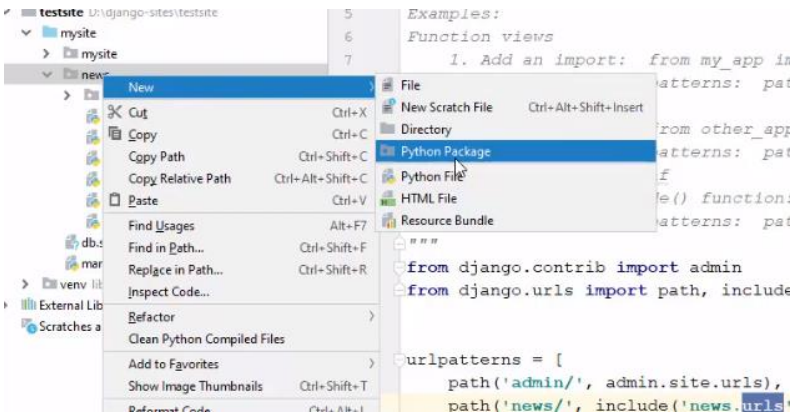


Рисунок 33

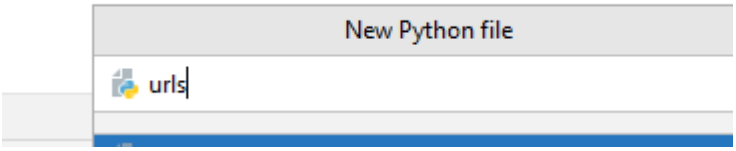


Рисунок 34

Импортируем функцию path



Рисунок 35

И создаем переменную urlpatterns со списком адресов.

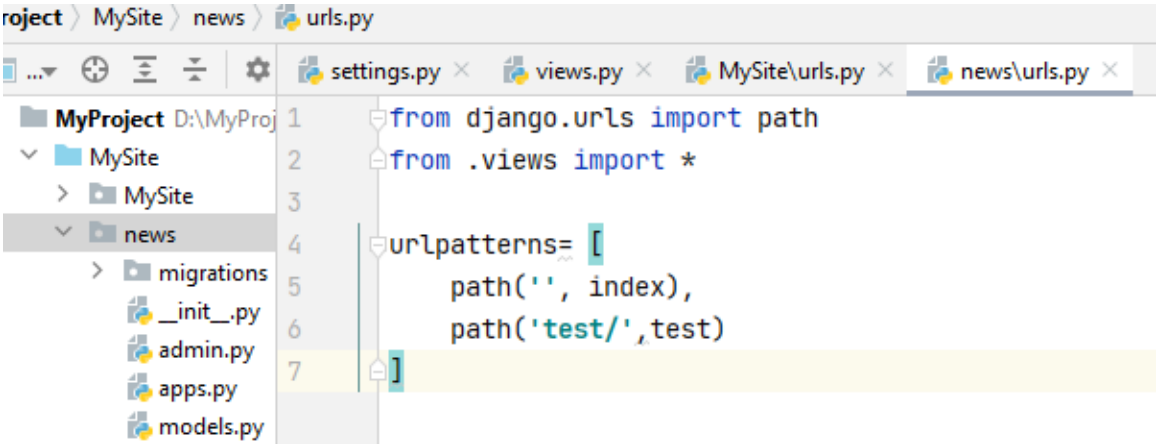


Рисунок 36



Рисунок 37

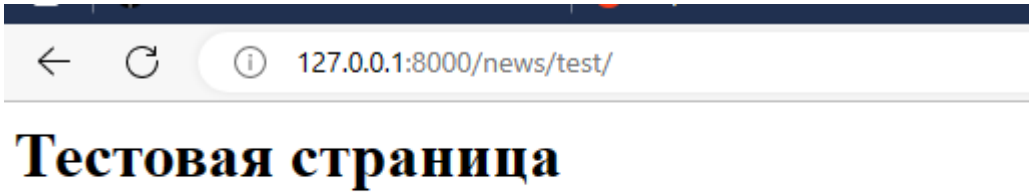


Рисунок 38